

sqli



Specification Driven Development

Développer à l'ère des LLMs

Philippe Bousquet – Architecte & Leader Java Community France
Anthony Baudoui – Architecte & CTO Nantes

Qui sommes-nous ?



Philippe Bousquet

- Architecte & Leader Java Community France
- Passionné de dev depuis mes 9 ans
- Près de 30 d'expérience professionnelle
- Pilote la Veille & l'Innovation Java

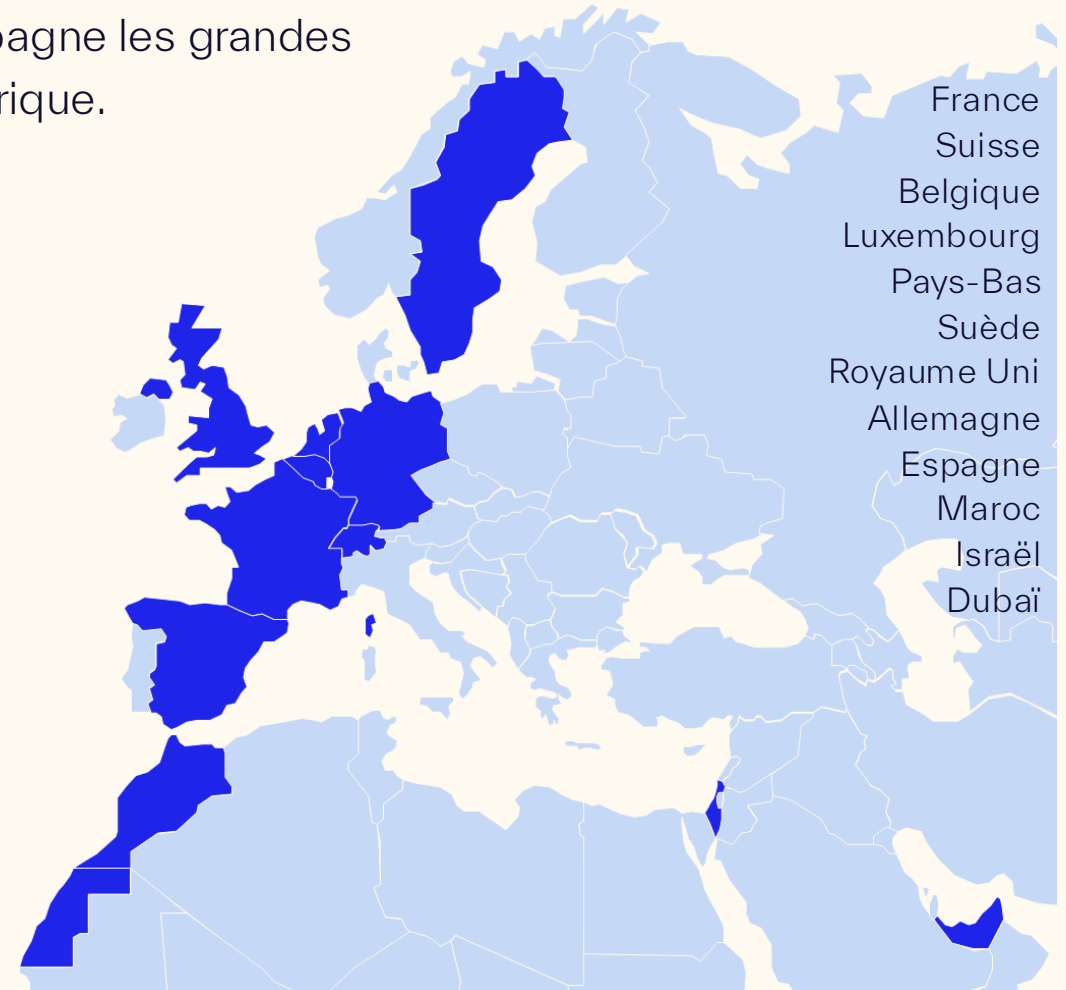
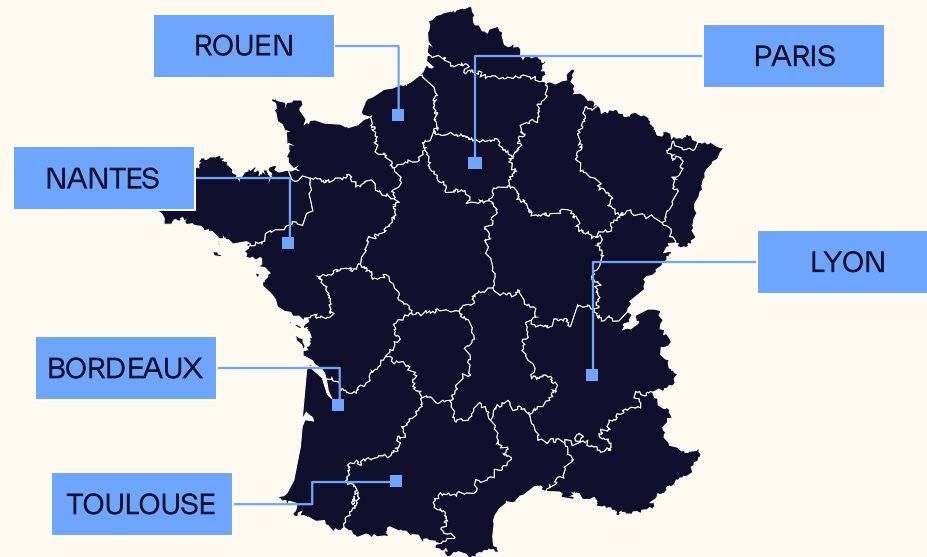


Anthony Baudouin

- Architecte & CTO Nantes
- ...

Le Groupe SQLI

SQLI est un groupe européen de services numériques qui accompagne les grandes marques internationales dans la création de valeur grâce au numérique.



1990

Création du groupe

253

M€ en 2025

2200

Employés

12

Pays

Specification Driven Development

Un peu de théorie



Assistants IA : de l'autocomplétion à...



Autocomplétion

"Je tape, l'IA complète."

- Suggestions inline
- Gain micro-productivité
- Dev "pilotage manuel"

Agents

"Je délègue des tâches."

- Génération multi-fichiers
- Tests / Docs / Refacto
- Boucles itératives

Vibe Coding

"Je décris, ça code."

- Très rapide pour prototyper
- Mais... intentions implicites
- Risque : dérive / incohérences

Specification-Driven Development



Concevez des logiciels à partir d'intentions explicites, et non à partir de suppositions.

Définition :

Le développement piloté par les spécifications est une approche dans laquelle des **spécifications claires et structurées** deviennent la **principale source de vérité**, et le **code est généré** en conséquence, et non comme point de départ.

Changer de rôle

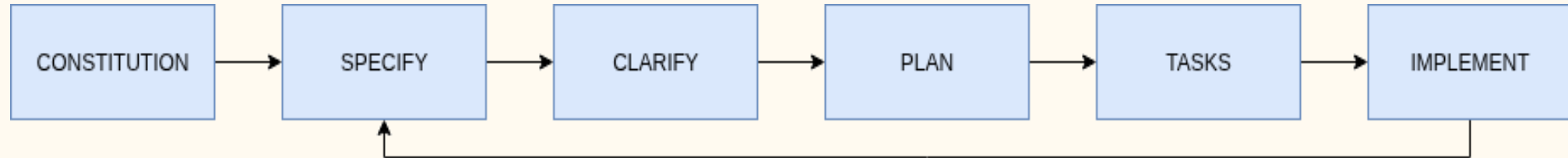
De développeur assisté... à ingénieur augmenté

Github Copilot :

- Compréhension du métier et de l'architecture
- Génération structurée (tests, docs, schémas)
- Détection proactive des incohérences
- Support clé de la phase de clarification

Spec-Kit

Un framework qui discipline l'IA



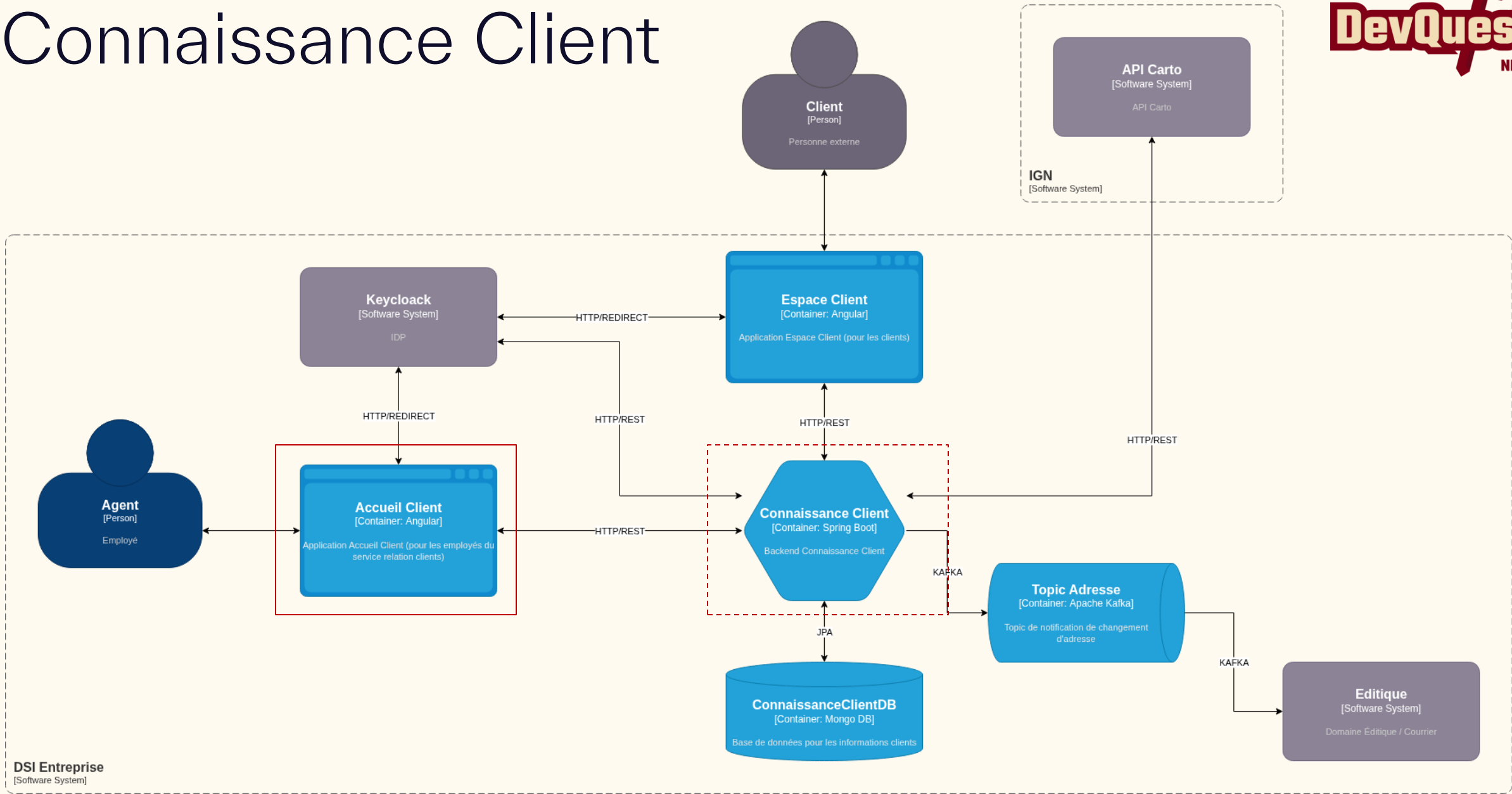
- Constitution : règles & architecture
- Spécification : intention fonctionnelle
- Clarification : éliminer les ambiguïtés
- Plan & tâches : livrer de façon prédictible
- Implémentation : génération itérative

Notre champ d'expérimentation

Connaissance Client



Connaissance Client



Les prérequis

Ce dont vous aurez besoin

- Git
- Un IDE avec Github Copilot (ou un autre assistant IA)
- UV (<https://docs.astral.sh/uv/getting-started/installation/>)
- Java 21, Maven
- React ou Angular, NPM
- Des connaissances en développement (préférable)

Récupérer le Repository



<https://github.com/darken33/specification-driven-development>

Un contrat OpenApi

connaissance-client-api.yaml

The image shows the Swagger Editor interface. On the left, the OpenAPI specification is displayed in a code editor. It defines an API titled 'Connaissance Client API' with version '1.0.0' and 'OAS 3.0'. The API is documented in French and describes operations for managing 'Connaissance Client' records. The specification includes a 'servers' section with two endpoints: 'http://localhost:8080/' (DEV Server) and 'https://jenesaispas/' (Azure Cloud). The 'paths' section defines several endpoints: 'GET /v1/connaissance-clients' (Liste des fiches de connaissance client), 'POST /v1/connaissance-clients' (Accès en écriture à une fiche de connaissance client), 'GET /v1/connaissance-clients/{id}' (Accès en lecture à une fiche de connaissance client), 'DELETE /v1/connaissance-clients/{id}' (Suppression d'une fiche de connaissance client), 'PUT /v1/connaissance-clients/{id}/adresse' (Changement d'adresse du client), and 'PUT /v1/connaissance-clients/{id}/situation' (Changement de situation du client). The 'components' section defines schemas for 'ConnaissanceClient' and 'ApiErrorResponse'.

Swagger Editor

Connaissance Client API 1.0.0 OAS 3.0

Documentation de l'API Connaissance Client

SQLI - Website
Send email to SQLI
Copyright (c) SQLI

Servers

http://localhost:8080/ - DEV Server

Authorize

ConnaissanceClient opérations de lecture / écriture sur les fiches de connaissance client

GET /v1/connaissance-clients Liste des fiches de connaissance client

POST /v1/connaissance-clients Accès en écriture à une fiche de connaissance client

GET /v1/connaissance-clients/{id} Accès en lecture à une fiche de connaissance client

DELETE /v1/connaissance-clients/{id} Suppression d'une fiche de connaissance client

PUT /v1/connaissance-clients/{id}/adresse Changement d'adresse du client

PUT /v1/connaissance-clients/{id}/situation Changement de situation du client

Schemas

ApiErrorResponse

Des maquettes

maquette_accueil-client.png

👤 Accueil Client

Liste des clients

+ Nouveau client

BP Bousquet
Philippe
📍 33800 — Bordeaux
[Voir le détail →](#)

CD Cesar Ceballos
Dulce
📍 33800 — Bordeaux
[Voir le détail →](#)

BT Bousquet
Thierry
📍 33600 — Pessac
[Voir le détail →](#)

BA Bousquet
Anne
📍 33600 — Pessac
[Voir le détail →](#)

CD Couderc
Damien
📍 31000 — Toulouse
[Voir le détail →](#)

Accueil Client

Créer une nouvelle application (Frontend)

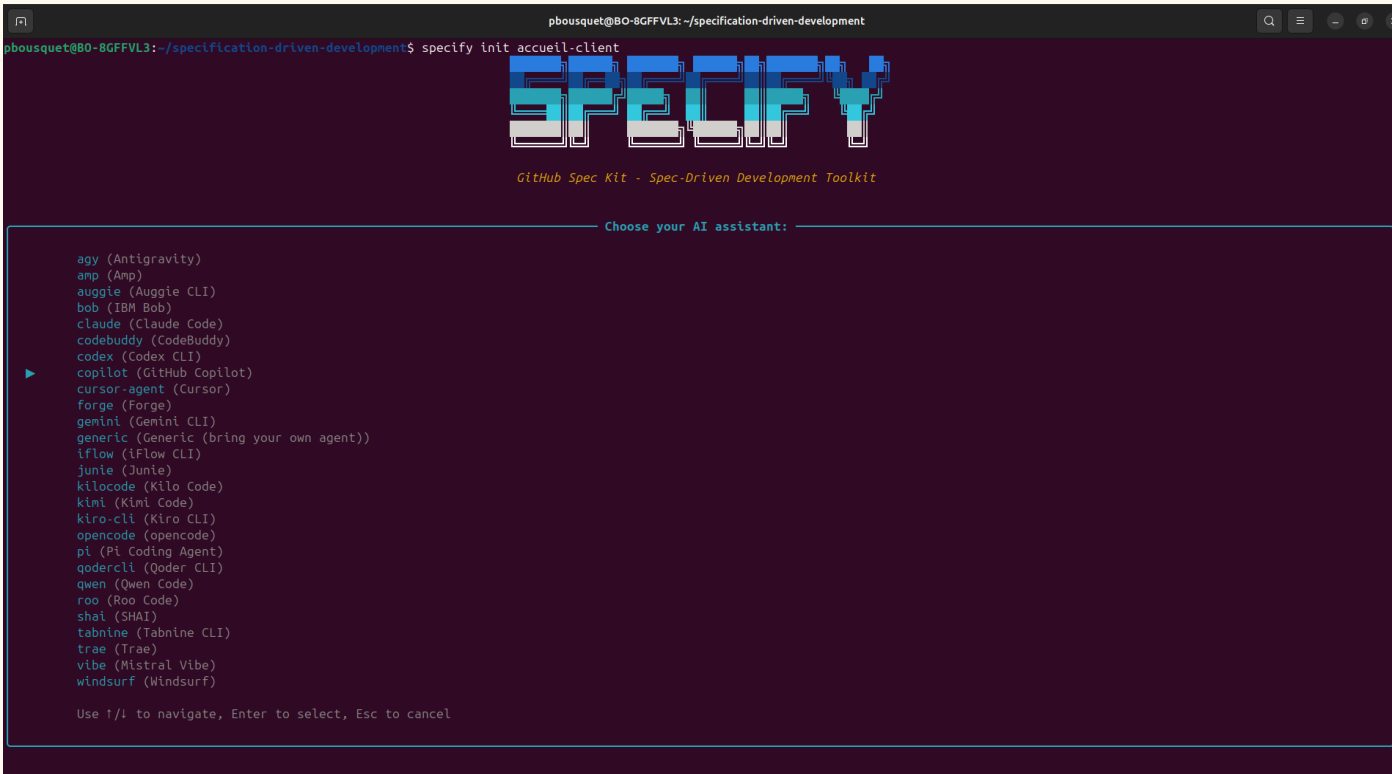


Initialisation de Spec-Kit

Via la CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
specify init accueil-client
```



```
pbousquet@BO-8GFFVL3: ~/specification-driven-development
pbousquet@BO-8GFFVL3:~/specification-driven-development$ specify init accueil-client

          SPECIFY
          GitHub Spec Kit - Spec-Driven Development Toolkit

          Choose your AI assistant:

  agy (Antigravity)
  amp (Amp)
  auggie (Auggie CLI)
  bob (IBM Bob)
  claude (Claude Code)
  codebuddy (CodeBuddy)
  codex (Codex CLI)
  ▶ copilot (GitHub Copilot)
  cursor-agent (Cursor)
  forge (Forge)
  gemini (Gemini CLI)
  generic (Generic (bring your own agent))
  iflow (iFlow CLI)
  junie (Junie)
  kllocode (Kilo Code)
  klm1 (Klm1 Code)
  kiro-cli (Kiro CLI)
  opencode (opencode)
  pi (Pi Coding Agent)
  qodercli (Qoder CLI)
  qwen (Qwen Code)
  roo (Roo Code)
  shai (SHAI)
  tabnine (Tabnine CLI)
  trae (Trae)
  vibe (Mistral Vibe)
  windsurf (Windsurf)

  Use !/i to navigate, Enter to select, Esc to cancel
```

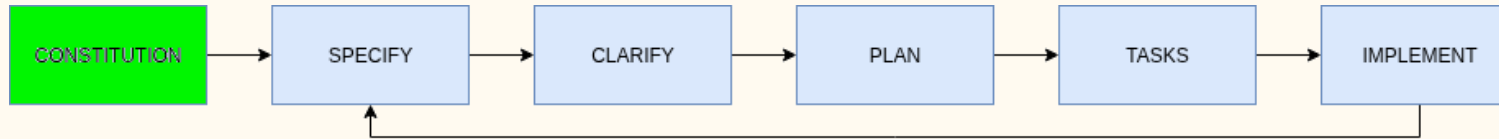


DevQuest
NIORT

```
✓ ACCUEIL-CLIENT
  ✓ .github
    ✓ agents
      ≡ speckit.analyze.agent.md
      ≡ speckit.checklist.agent.md
      ≡ speckit.clarify.agent.md
      ≡ speckit.constitution.agent.md
      ≡ speckit.implement.agent.md
      ≡ speckit.plan.agent.md
      ≡ speckit.specify.agent.md
      ≡ speckit.tasks.agent.md
      ≡ speckit.taskstoissues.agent.md
    > prompts
  ✓ .specify
    > scripts
    > templates
    > .vscode
```

Demo / VS Code

Constitution



Exemple de prompt :

```
/speckit.constitution En tant qu'architecte technique spécialisé en Angular et TypeScript, définis la structure, les principes architecturaux et de développement d'une application front-end moderne. Pour ce faire, suis les meilleures pratiques actuelles, en respectant au minimum les principes suivants :
```

- * Utiliser les dernières versions d'Angular et de TypeScript.
- * Principe KISS : Privilégier la simplicité.
- * Séparer explicitement l'interface utilisateur du traitement.

S'agissant d'un POC, n'ayant pas pour but de partir en production, on ne traitera pas les éléments suivants :

- * Authentification et Sécurité
- * Pipelines de CI/CD

Définis uniquement le fichier constitution.md, puis me le retourner afin que je puisse vérifier le respect de ces principes.

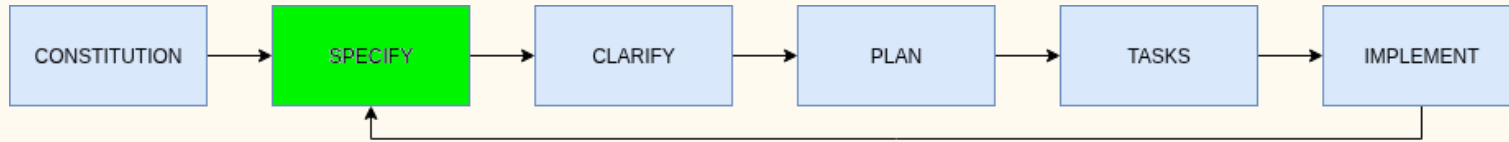


Principes attendus :

- Angular & TypeScript à jour
- KISS (Keep It Simple, Stupid)
- Séparation UI vs Traitement
- Mode POC

Demo / VS Code

Spécifications



Exemple de prompt :

```
/speckit.specify Nous disposons actuellement d'une application backend exposant des API REST définies dans le contrat OpenAPI (fichier `connaissance-client-api.yaml`). En tant qu'architecte technique et fonctionnel possédant de solides compétences en UX/UI, Tu dois spécifier une application frontend qui accède à tous les points de terminaison du contrat OpenAPI.
```

Nous disposons des maquettes suivantes :

- * Page d'accueil / liste des clients `maquette\_accueil-client.png`.
- * Page nouveau client `maquette\_nouveau-client.png`.
- * Page détail client `maquette\_detail-client.png`.
- * Pop Up modification d'adresse `maquette\_modifier-adresse.png`.
- * Pop Up modification situation `maquette\_modifier-situation.png`.
- * Pop Up supprimer client `maquette\_supprimer-client.png`.

Une première version MVP consiste à disposer de la page d'accueil, le backend API n'étant pas disponible il faudra disposer d'un mock pour les tests.

Une fois la tâche réalisée, rends moi la main afin que je valide ce que tu auras rédigé.



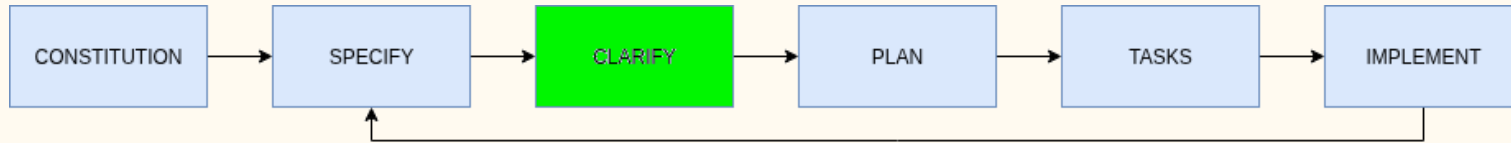
DevQuest
NIORT

Spécification attendue :

- Une US / Maquette – endpoint API
- Un Mock service API
- Une Architecture
- Un MVP
- Diagramme de séquence ?

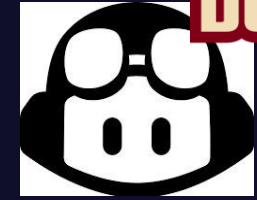
Demo / VS Code

Clarification



Exemple de prompt :

```
/speckit.clarify Clarifies la spécification ci-jointe si nécessaire.
```



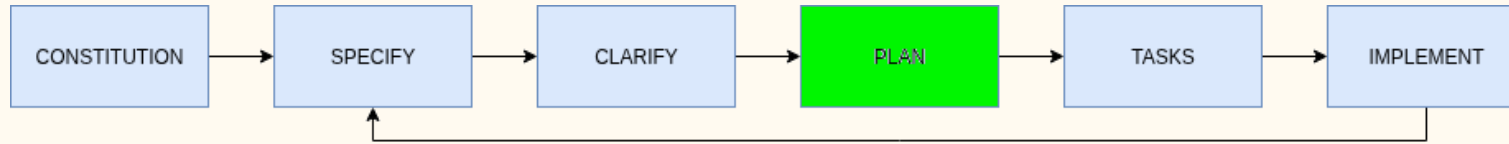
DevQuest
NIORT

Passage en mode interactif

- Liste vide ?
- Mock Dataset ?
- Spinner ?
- Contrôles de surface ?
- Couleurs des cartes ?

 Demo / VS Code

Définir le plan



Exemple de prompt :

```
/speckit.plan Crées un plan pour la spécification ci-jointe. Une fois le plan établi rends moi la main afin que je le valide.
```



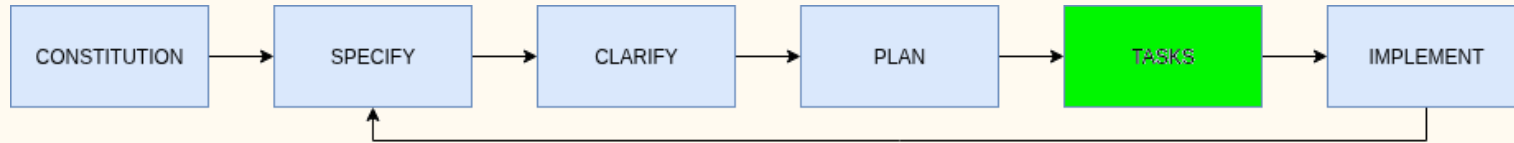
DevQuest
NIORT

Plan défini

- Phase 1: Angular project setup
- Phase 2: Foundation (mock API + models)
- Phases 3-7: User stories
- Phase 8: Polish & testing

 Demo / VS Code

Transformer le plan en tâches



Exemple de prompt :

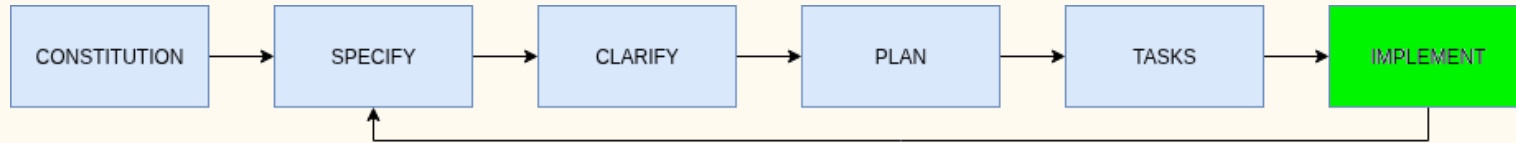
```
/speckit.tasks Transforme le plan ci-joint en tâches. Une fois les tâches établies rends moi la main afin que je les valide.
```



Tâches définies

- 8 phases
- N tâches

Démarrer l'implémentation



Exemple de prompts :

```
/speckit.implement Implémente la phase 1, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu initial.
```

```
/speckit.implement Lance l'implémentation par phase à partir de la liste des tasks, en suivant le plan et les spec.
```

Améliorer l'interface utilisateur :

```
/speckit.implement Parfait cela fonctionne, par contre le design n'est pas excellent, peux-tu me proposer un bandeau design pour le titre, et plus de couleur sur les cartes clients ?.
```



DevQuest
NIORT

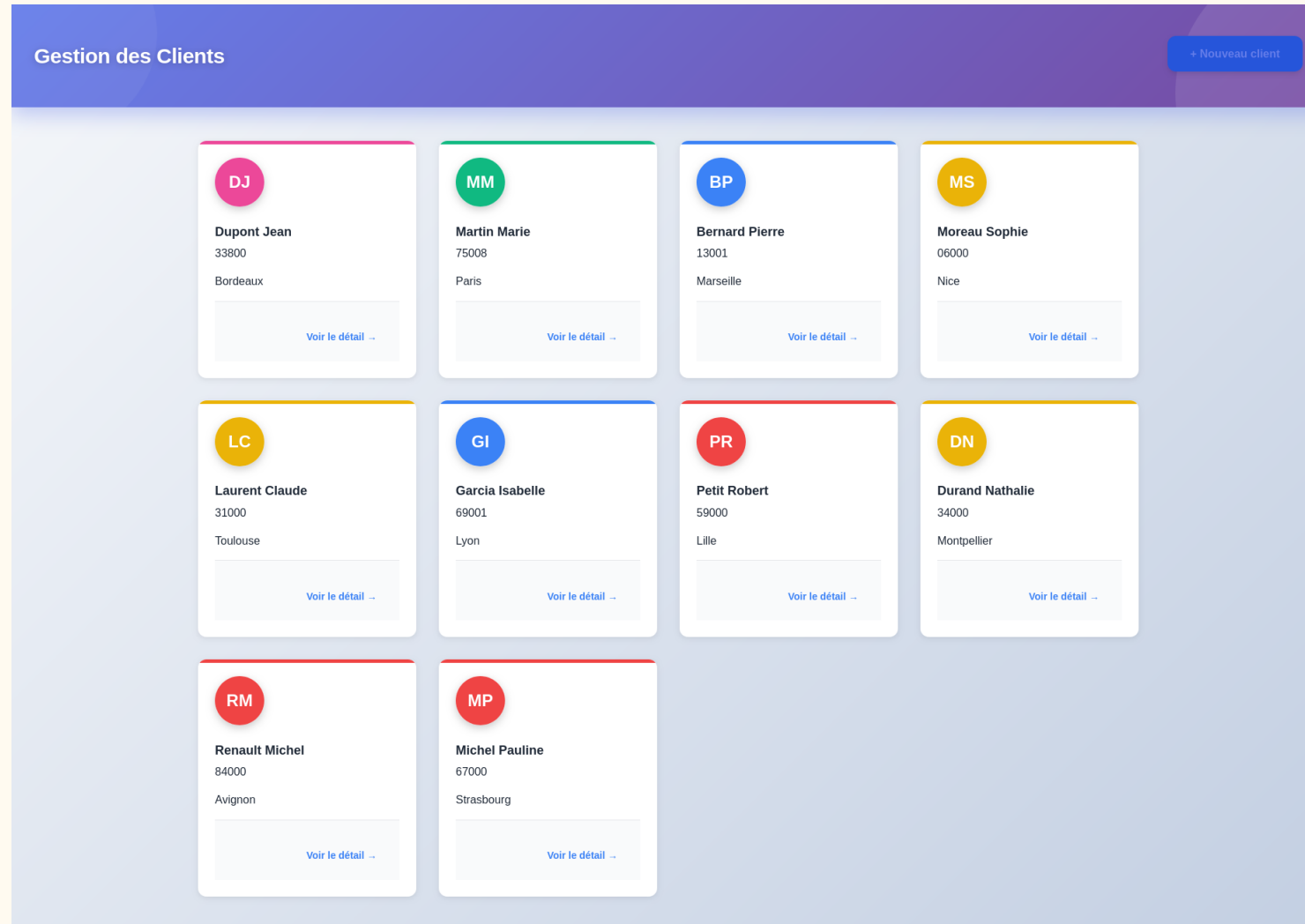
Passage en cycle itératif

- Phase 1 -> Setup
- Phase 2 -> Fondations
- Phase 3 -> User Story 1
- ...

Correctifs si nécessaire

 Demo / VS Code

Notre application générée



Connaissance Client

Reprise d'une application existante (Backend API)

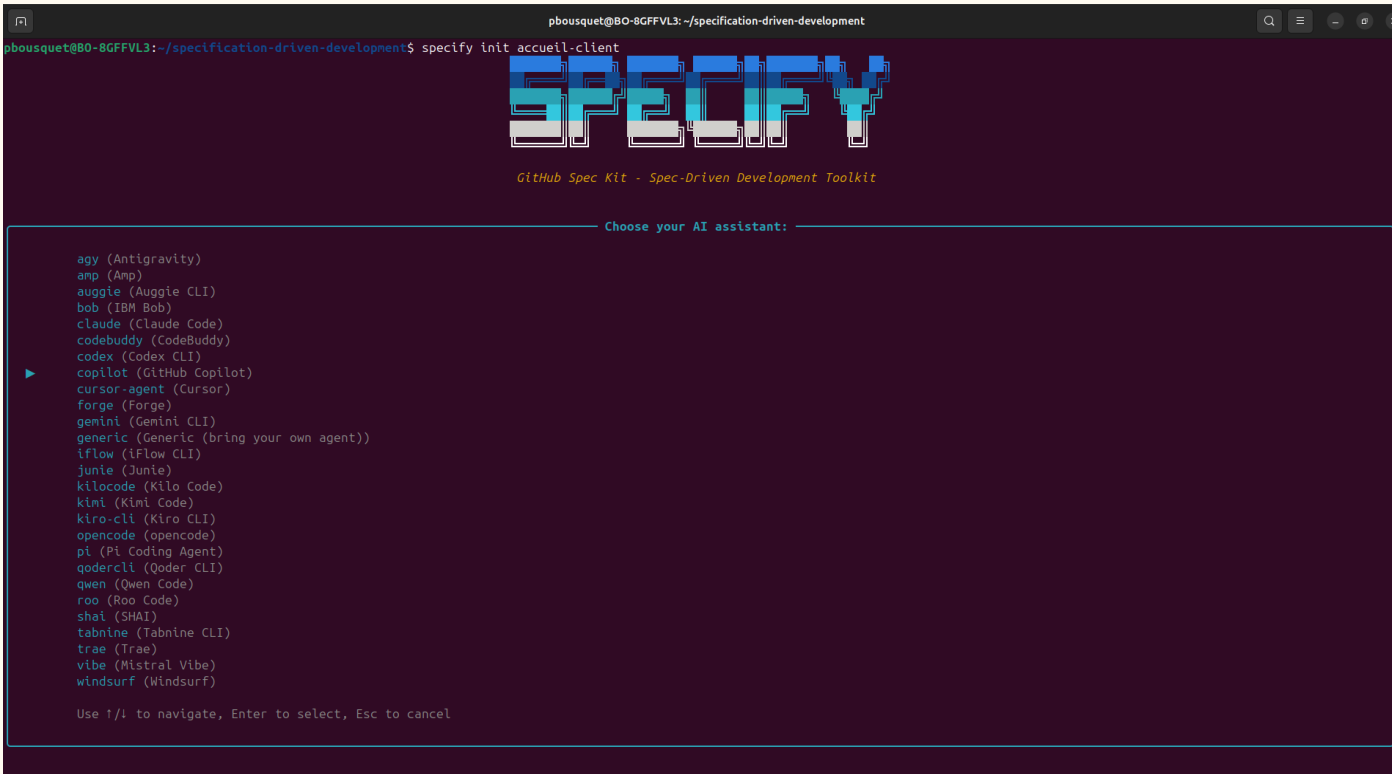


Initialisation de Spec-Kit

Via la CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
cd connaissance-client && specify init --here
```



```
pbousquet@BO-8GFFVL3: ~/specification-driven-development
pbousquet@BO-8GFFVL3:~/specification-driven-development$ specify init accueil-client

          SPECIFY
          GitHub Spec Kit - Spec-Driven Development Toolkit

          Choose your AI assistant:

  agy (Antigravity)
  amp (Amp)
  auggie (Auggie CLI)
  bob (IBM Bob)
  claude (Claude Code)
  codebuddy (CodeBuddy)
  codex (Codex CLI)
  ▶ copilot (GitHub Copilot)
  cursor-agent (Cursor)
  forge (Forge)
  gemini (Gemini CLI)
  generic (Generic (bring your own agent))
  iflow (iFlow CLI)
  junie (Junie)
  kllocode (Kilo Code)
  klm1 (Klm1 Code)
  kiro-cli (Kiro CLI)
  opencode (opencode)
  pi (Pi Coding Agent)
  qodercli (Qoder CLI)
  qwen (Qwen Code)
  roo (Roo Code)
  shai (SHAI)
  tabnine (Tabnine CLI)
  trae (Trae)
  vibe (Mistral Vibe)
  windsurf (Windsurf)

  Use !/i to navigate, Enter to select, Esc to cancel
```

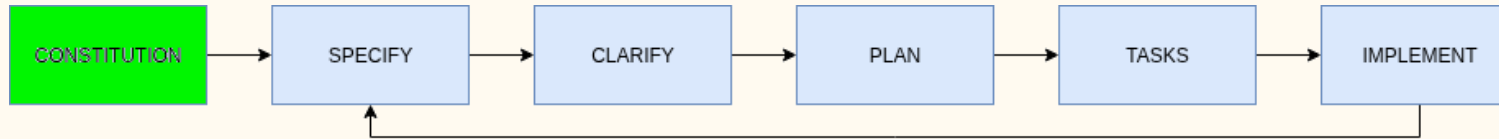


DevQuest
NIORT

```
▼ CONNAISSANCE-CLIENT
  ▼ .github
    ▼ agents
      ≡ speckit.analyze.agent.md
      ≡ speckit.checklist.agent.md
      ≡ speckit.clarify.agent.md
      ≡ speckit.constitution.agent.md
      ≡ speckit.implement.agent.md
      ≡ speckit.plan.agent.md
      ≡ speckit.specify.agent.md
      ≡ speckit.tasks.agent.md
      ≡ speckit.taskstoissues.agent.md
    > prompts
    > .mvn/wrapper
  ▼ .specify
    > scripts
    > templates
    > .vscode
```

Demo / VS Code

Constitution



Exemple de prompt :

```
/speckit.constitution En tant qu'architecte technique spécialisé en Java et Spring-Boot, effectue une analyse complète du projet afin d'en déterminer les principes d'architecture et les pratiques de développements afin d'alimenter le constitution.md. Une fois la constitution rédigée, rends-moi la main afin que je procède à sa validation avant de passer à l'étape suivante.
```

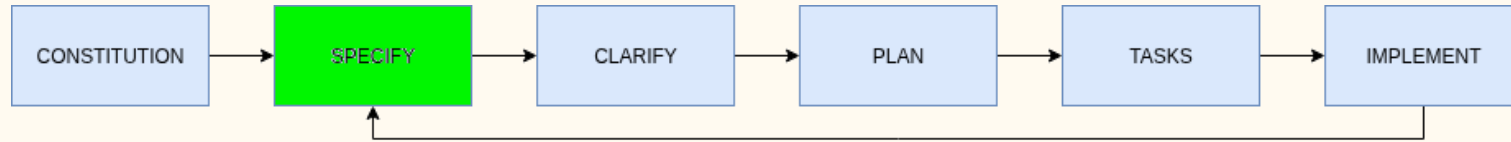


Principes attendus :

- Domain Driven Design
- Architecture Hexagonale
- API-First Contract
- TDD
- Java 21, Spring Boot 4

 Demo / VS Code

Rétro-Spécification



Exemple de prompt :

```
/speckit.specify En tant qu'architecte technique et fonctionnel  
spécialisé en Java et Spring-Boot, effectue une analyse complète du code  
de ce projet afin d'en faire une rétro spécification, appuies toi en  
premier lieu sur les contrats OpenApi, AsyncApi et Les Modèles de  
données. Effectue également un schéma d'architecture pour présenter les  
interactions entre les différents composants. Rends-moi la main, afin  
que je procède à sa validation avant de passer à l'étape suivante.
```



DevQuest
NIORT

Spécification attendue :

- Entité clé: Client
- Endpoints principaux:
 - GET (liste)
 - POST (creation)
 - GET {id}
 - DELETE {id}
 - PUT {id}/adresse
 - PUT {id}/situation
- Validation adresse via IGN.
- PUT adresse -> Kafka
- Persistance: MongoDB

Demo / VS Code

D'autres fonctionnalités ?



Laissez libre court à votre imagination

Exemples Backend

- Ajouter un numéro de téléphone
- Ajouter un email
- Mettre en place le contrôle JWT
- ...

Exemples Frontend

- Mettre en place les autres écrans
- Mettre en place un Logo
- Mettre en place l'authentification
- ...

Conclusion

À retenir



À retenir

- Spec-Kit, un workflow simple et structuré pour le Specification-Driven Development
- Essayez-le, testez-le, pratiquez-le : c'est en l'utilisant qu'il prend tout son sens
- Commencez petit : le [String Calculator Kata](#) est un excellent point d'entrée
- Attention aux limites : les technos sorties après la date d'entraînement du LLM
- Le **rétro-engineering** fonctionne **très bien** sur des applications bien architecturées (Hexagonal, DDD, etc.)
- **Spécifiez en langage maternel** : vous serez souvent plus précis
- Pour mieux spécifier, **pensez design-first** : **OpenAPI, AsyncAPI, Data Models** avant le code
- **TOUJOURS VÉRIFIER CE QUE GÉNÈRE VOTRE IA**

sqli



Thank You



Elevate. Digitally